

The following table contains the basic elements, along with description and examples.

regex	Description	Example
<code>^</code>	The start-of-line marker.	<code>^tux</code> matches any line that starts with <i>tux</i> .
<code>\$</code>	The end-of-line marker.	<code>tux\$</code> matches any line that ends with <i>tux</i> .
<code>.</code>	Matches any one character.	<i>Hack.</i> matches <i>Hack1</i> , <i>Hacki</i> but not <i>Hack12</i> , <i>Hackil</i> ; only one additional character matches.
<code>[]</code>	Matches any one of the character set inside <code>[]</code> .	<code>coo[kl]</code> matches <i>cook</i> or <i>cool</i> .
<code>[^]</code>	Exclusion set; the carat negates the set of characters in the square brackets; text matching this set will not be returned as a match.	<code>9[^01]</code> matches <i>92</i> , <i>93</i> but not <i>91</i> and <i>90</i> .
<code>[-]</code>	Matches any character within the range specified in <code>[]</code> .	<code>[1-5]</code> matches any digits from <i>1</i> to <i>5</i> .
<code>?</code>	The preceding item must match one or zero times.	<code>colour?r</code> matches <i>color</i> or <i>colour</i> but not <i>colourr</i> .
<code>+</code>	The preceding item must match one or more times.	<code>Rollno-9+</code> matches <i>Rollno-99</i> , <i>Rollno-9</i> but not <i>Rollno-</i> .
<code>*</code>	The preceding item must match zero or more times.	<code>co*l</code> matches <i>cl</i> , <i>col</i> , <i>cool</i> .
<code>()</code>	Creates a sub-string in the regex match.	Explained below, in the section 'Sub-string match and back-referencing'.
<code>{n}</code>	The preceding item must match exactly <i>n</i> times.	<code>[0-9]{3}</code> matches any three-digit number. This can be expanded as: <code>[0-9][0-9][0-9]</code> .
<code>{n,}</code>	Minimum number of times that the preceding item should match.	<code>[0-9]{2,}</code> matches any number that is two digits or more in length.

<code>{n, m}</code>	Specifies the minimum and maximum number of times the preceding item should match.	<code>[0-9]{2,5}</code> matches any number that is between two and five digits in length.
<code> </code>	Alternation—one of the items on either side of <code> </code> should match.	<code>Oct (1st 2nd)</code> matches <i>Oct 1st</i> or <i>Oct 2nd</i> .
<code>\</code>	The escape character for escaping any of the special characters given above.	<code>a\b</code> matches <i>a.b</i> but not <i>ajb</i> . The dot is not interpreted as the special 'match any one character' regex shown above, but instead a literal dot (period) ASCII character is sought to be matched. Another example: if you're searching for the US currency symbol '\$', and not the end-of-line marker, you must precede it with a back-slash, like this: <code>\\$</code>

There are a few character classes, called POSIX classes, in the format `[[:name:]]` that can be conveniently used, instead of spelling out the character set each time. Note that, as shown in the example column, you need to enclose the class itself in another pair of square brackets. For example:

```
$ echo -e "max\nOR\nMatrix" | sed '/[[:alpha:]]/d'
OR
$ echo -e "max\nOR\nMatrix" | sed '/[[:alpha:]]/d'
$
```

In the first case, the set is interpreted literally—the words '*max*' and '*matrix*' are deleted because they contain '*a*', one of the letters in the character set. In the second command, with another pair of square brackets around the class, all input lines are deleted, because all lines contain (at least one) alphabet.

Regex	Description	Example
<code>[[:alnum:]]</code>	Alphanumeric characters	<code>[[:alnum:]]+</code>
<code>[[:alpha:]]</code>	Alphabet character (lower-case and upper-case)	<code>[[:alpha:]]{4}</code>
<code>[[:blank:]]</code>	Space and tab	<code>[[:blank:]]*</code>